Topic 10 Chapter 12 Worksheet

In the worksheets for this class, you are going to create MySQL statement for each task described below.

You are required to take a screen shot of the SQL statements and the output for each task.

Paste the screenshot of your results under the numbered question.

Save your work and upload the document to the assignment folder by the due date in the Syllabus.

Use the Oracle tutorial database.

Set AUTOCOMMIT to 0 so you can roll back all your changes.

1. Create a view of the employees table.

Limit the view to just employees whose first name starts with a S or T. Sort the results by last name, descending.

SELECT from the VIEW and display the results.

```
3     -- 1. Create a view of the employees table
4 •   CREATE VIEW view_employees_s_t AS
5         SELECT employee_id, first_name, last_name
6         FROM employees
7         WHERE first_name REGEXP "^[S,T]"
8         ORDER BY last_name DESC;
9
10 •  SELECT *
11    FROM view_employees_s_t;
```

Result Grid | Filter Rows: | Export: | Wrap Ce

employee_id	first_name	last_name
67	Sienna	Simpson
38	Sonny	Russell
53	Sophia	Reynolds
11	Tyler	Ramirez
107	Summer	Payne
52	Sophie	Owens
72	Sofia	Hicks
100	Thea	Hawkins
57	Scarlett	Gibson
34	Seth	Foster
1	Tommy	Bailey

2. Create a view from the order_items table.

Multiply the quantity by the unit_price and name the column "total". Limit the output to the top 15 totals.

SELECT from the VIEW and display the results.

```sql
-- 2. Create a view from the order_items table.
-- Multiply the quantity by the unit_price and name the column "total".
-- Limit the output to the top 15 totals.
CREATE VIEW view_top_15_order_items_total AS
    SELECT order_id, item_id, quantity * unit_price AS Total
    FROM order_items
    ORDER BY Total DESC
    LIMIT 15;

SELECT *
FROM view_top_15_order_items_total;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

order_id	item_id	Total
70	6	603023.3200
78	6	600155.0000
14	2	550487.0000
32	2	389855.7000
46	8	368583.0400
54	2	367198.6400
92	4	354419.6700
75	4	344872.3500
59	4	328038.4200
78	1	323400.0000
18	4	318658.5900
104	6	315528.6100
67	9	311878.6200
90	4	309041.1200
40	5	307683.5100

3. Create a view of joined tables.

Join warehouses to locations to countries to get the list of where the warehouses are.

SELECT from the VIEW and display the results.

```sql
-- 3. Create a view of joined tables.
-- Join warehouses to locations to countries
-- to get the list of where the warehouses are.
CREATE VIEW view_warehouse_locations AS
    SELECT w.warehouse_name, l.city, l.state, c.country_name
    FROM warehouses w
    JOIN locations l USING (location_id)
    JOIN countries c USING (country_id);

SELECT *
FROM view_warehouse_locations;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouse_name	city	state	country_name
Southlake, Texas	Southlake	Texas	United States of America
San Francisco	South San Francisco	California	United States of America
New Jersey	South Brunswick	New Jersey	United States of America
Seattle, Washington	Seattle	Washington	United States of America
Toronto	Toronto	Ontario	Canada
Sydney	Sydney	New South Wales	Australia
Mexico City	Mexico City	Distrito Federal,	Mexico
Beijing	Beijing	NULL	China
Bombay	Bombay	Maharashtra	India

4. Using the view, update the city of the Bombay warehouse to "North Bombay".

```sql
-- 4. Using the view,
-- update the city of the Bombay warehouse to "North Bombay".
UPDATE view_warehouse_locations
SET city = "North Bombay"
WHERE city = "Bombay";

SELECT *
FROM view_warehouse_locations;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

warehouse_name	city	state	country_name
Southlake, Texas	Southlake	Texas	United States of America
San Francisco	South San Francisco	California	United States of America
New Jersey	South Brunswick	New Jersey	United States of America
Seattle, Washington	Seattle	Washington	United States of America
Toronto	Toronto	Ontario	Canada
Sydney	Sydney	New South Wales	Australia
Mexico City	Mexico City	Distrito Federal,	Mexico
Beijing	Beijing	NULL	China
Bombay	North Bombay	Maharashtra	India

5. Create a view that calculates the profit by product name from the products table.

Profit is the list price minus the cost. Name the column "profit".

SELECT from the VIEW and display the results.

```
50      -- 5. Create a view that calculates the profit by product name from the products table.
51      -- Profit is the list price minus the cost. Name the column "profit".
52 •    CREATE OR REPLACE VIEW product_profit AS
53          SELECT product_name, (list_price - standard_cost) AS profit
54          FROM products;
55
56 •    SELECT *
57      FROM product_profit;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_name	profit
G.Skill Ripjaws V Series	190.63
Intel Xeon E5-2697 V4	410.59
Corsair CB-9060011-WW	226.48
AMD 100-505989	571.32
PNY VCQK6000-PB	550.48
Zotac ZT-P10810A-10P	147.64
G.Skill Ripjaws V Series	78.59
Intel Xeon E5-1650 V4	66.52
Intel Xeon E5-2640 V4	161.28
Crucial	231.66
PNY VCQP5000-PB	412.90
Gigabyte GV-N108TAOR...	228.93
G.Skill Ripjaws V Series	86.36

6. Using the view update the profit for the "Crucial" product to 1000.

What happened? Look up the error code and explain.

```
60 •    UPDATE product_profit
61      SET profit = 1000
62      WHERE product_name = "Crucial";
```

The update query failed and generated an error: Error Code: 1348. Column 'profit' is not updatable
The “profit” column in the “product_profit” view is a calculated expression, not a real physical column in the “products” base table. Because profit is derived from a calculation, MySQL cannot determine how to update the original columns (list_price or standard_cost), making this column read-only.

7. Create a view of the quantity in the warehouses.

Include the product_id, product_name, warehouse_id, quantity. Only show those quantities that are less than 5. Add a CHECK option.

SELECT from the VIEW and display the results.

```sql
-- 7. Create a view of the quantity in the warehouses.
-- Include the product_id, product_name, warehouse_id, quantity.
-- Only show those quantities that are less than 5.  Add a CHECK option.
CREATE OR REPLACE VIEW low_stock_warehouses AS
    SELECT i.product_id, p.product_name, w.warehouse_id, i.quantity
    FROM warehouses w
    JOIN inventories i USING (warehouse_id)
    JOIN products p USING (product_id)
    WHERE quantity < 5
WITH CHECK OPTION;

SELECT *
FROM low_stock_warehouses;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

product_id	product_name	warehouse_id	quantity
17	Corsair Vengeance LPX	9	3
18	Crucial	9	3
19	Intel Core i7-6950X (OEM/Tray)	9	3
20	Corsair Dominator Platinum	9	4
21	Corsair Dominator Platinum	9	4
47	Intel Xeon E5-2697 V2	9	0
101	Kingston HyperX Predator	9	3

8. Using the above view, try to update the "Crucial" product by changing the quantity to 10.

What happened and why?

```sql
-- 8. Using the above view, try to update the "Crucial" product by changing the quantity to 10.
-- What happened and why?
UPDATE low_stock_warehouses
SET quantity = 10
WHERE product_name = "Crucial";
```

Output

Action Output

#	Time	Action	Message
9088	15:27:11	UPDATE low_stock_warehouses SET quantity = 10 WHERE product_name = "Crucial"	Error Code: 1369. CHECK OPTION failed 'ot.low_stock_warehouses'

The UPDATE statement failed with Error Code: 1369. CHECK OPTION failed 'ot.low_stock_warehouses'

The view was defined with WHERE quantity < 5 and the WITH CHECK OPTION clause. The WITH CHECK OPTION ensures that any INSERT or UPDATE performed through the view must satisfy the view's WHERE condition. Updating the quantity to 10 violates this rule, so MySQL rejected the change.

9. Create a view that displays the region_id and region_name from the regions table.

Using the view, add Iceland, region_id=10 to the table.

```sql
-- 9. Create a view that displays the region_id and region_name from the regions table.
-- Using the view, add Iceland, region_id=10 to the table.
-- Using the view delete Iceland from the table.
CREATE VIEW view_regions AS
    SELECT region_id, region_name
    FROM regions;

INSERT INTO view_regions
    (region_id, region_name)
VALUES
    (10, "Iceland");

SELECT *
FROM view_regions;
```

Result Grid | Filter Rows: | Export: | Wrap Cell Content:

region_id	region_name
1	Europe
2	Americas
3	Asia
4	Middle East and Africa
10	Iceland

Using the view delete Iceland from the table.

```sql
DELETE FROM view_regions
WHERE region_id = 10;

SELECT *
FROM view_regions;
```

Result Grid | Filter Rows: | Export:

region_id	region_name
1	Europe
2	Americas
3	Asia
4	Middle East and Africa

10. Clean up your database

Did your ROLLBACK work or did you have to reload the script to build the database?

I had to reload the script to rebuild the database because ROLLBACK did not undo the changes.

Even though AUTOCOMMIT was set to 0, DDL (Data Definition Language) statements such as CREATE VIEW trigger an implicit COMMIT in MySQL. Running CREATE VIEW in subsequent questions automatically committed all prior pending transactions (such as the UPDATE of the Bombay warehouse to "North Bombay"). Because the modifications were permanently committed to the base tables, ROLLBACK had no effect.